



ENTI
Escola de Noves
Tecnologies Interactives



UNIVERSITAT DE
BARCELONA

“Estudi de complexitat - Tatami Honeypot”

Autor:

Martí Tarrasón, Lluç Galceran, Roger Vallés, Ian Nogueira

Assignatura:

Anàlisi i Disseny d'Algorismes Avançats

Curs:

Ciberseguretat, 2025-2026

Data d'entrega:

Barcelona, 28/05/2026

Index

Index.....	2
Introducció.....	2
Anàlisi de Complejitat Algorítmica.....	3
1. Cerca de patrons.....	3
2. Construcció del graf.....	4
3. Detecció de fases.....	5
4. Lectura de CSV.....	7
Benchmarks.....	8
Escalabilitat i límits.....	10
Propostes de optimització.....	11
Conclusions.....	12

Introducció

Tatami Honeypot és un sistema interactiu que simula un servidor Ubuntu 22.04 compromès mitjançant un model de màquina d'estats representat per un graf de fases. El sistema implementa una detecció dinàmica d'atacs basada tant en l'historial de comandes de l'usuari com en l'escaneig de patrons maliciosos coneguts.

Aquest document presenta l'anàlisi de complexitat algorísmica de les funcions crítiques del projecte, avalua el seu rendiment mitjançant proves empíriques, en determina els límits d'escalabilitat i proposa optimitzacions formals per a futures iteracions del sistema.

Anàlisi de complexitat algorítmica

1. Cerca de patrons

Ubicació: patrons.py

Propòsit: Detectar si una seqüència de comandes (patró d'atac) existeix com a subseqüència dins l'historial de comandes executades.

```
=====py=====
def caçar_patro(historial, seq_atac, i_hist=0, i_seq=0):
    if i_seq == len(seq_atac):
        return True # Patró trobat
    if i_hist == len(historial):
        return False # Historial esgotat

    if seq_atac[i_seq] in historial[i_hist]:
        return caçar_patro(historial, seq_atac, i_hist + 1, i_seq + 1)

    return caçar_patro(historial, seq_atac, i_hist + 1, i_seq)
=====
```

Complexitat teòrica:

$$T(n, m) = O(n \times m)$$

On:

N = longitud de l'historial de comandes

M = longitud del patró a cercar

La funció utilitza cerca recursiva de subseqüència. En el pitjor cas, la funció recorre cada comanda de l'historial (n iteracions) i per a cadascuna potencialment compara tots els elements del patró (m comparacions). Cada crida recursiva processa exactament un element de l'historial o patró sense processament duplicat.

Casos de Complexitat

Tipus	Complexitat	Descripció
Millor cas	$O(1)$	El primer comanda coincideix completament
Cas mitjà	$O(n)$	El patró es troba a la meitat
Pitjor cas	$O(n \times m)$	El patró no existeix, explorament complet



Complexitat espacial

$$S(m) = O(m)$$

Causada per la pila de recursió, amb profunditat màxima igual a $\max(n, m)$.

Exemple pràctic

Historial: [whoami, sudo -l, cat /etc/passwd, id]" (4 elements)

Patró: "[sudo, cat]" (2 elements)

Execució:

- Comparació 1: "sudo" $\in$ "whoami"? No $\rightarrow$ avanç historial
- Comparació 2: "sudo" $\in$ "sudo -l"? Sí $\rightarrow$ avanç patró
- Comparació 3: "cat" $\in$ "cat /etc/passwd"? Sí $\rightarrow$ avanç patró
- Comparació 4: "seq_atac" completat $\rightarrow$ Retorna True
-

Total operacions: 4 (cas favorable).

Optimitzacions possibles

Aho-Corasick: Reduir multiple pattern matching de $O(n \times m \times k)$ a $O(n + m + z)$ on z és el nombre de coincidències

Boyer-Moore-Horspool: Per a cadenes individuals, salt de caracters mala coincidència

Memoization: Caché resultats de subsequències ja calculades

2. Construcció del graf

Ubicació: grafo.py - Línia 11

Propòsit: Carregar les fases des de fases.csv, les transicions des de transicions.csv i construir l'estructura de graf (nodes + aristes) en memòria.

Pseudocodi

```
=====
def muntar_xarxa_paranoid():
    fases = {}

    # Llegir fases.csv: O(f) on f = nombre de fases
    with open(fases_csv) as f:
        for row in csv.DictReader(f):
            fases[int(row["id"])] = NodoFase(
                row["nombre"],
                int(row["riesgo"])
            )

    # Llegir transicions.csv: O(t) on t = nombre de transicions
    with open(transicions_csv) as f:
```

```
for row in csv.DictReader(f):
    origen = int(row["desde"])
    destino = int(row["hasta"])
    fases[origen].rutes[row["clave"]] = fases[destino]
```

```
return fases[0], fases
```

Complexitat teòrica

$$T(f, t) = O(f + t)$$

On:

f = nombre de fases en "fases.csv"

t = nombre de transicions en "transicions.csv"

Justificació

La funció realitza dues passades independents i seqüencials:

Primera passada: Iterar sobre cada fila de "fases.csv" $\rightarrow O(f)$ operacions

Segona passada: Iterar sobre cada fila de "transicions.csv" $\rightarrow O(t)$ operacions

Les operacions internes (assignacions, accés a diccionari) són $O(1)$ amortitzat. No hi ha bucles anidats ni cerques iteratives.

Casos de Complexitat

Tipus	Complejitat	Descripció
Millor/Mitjà/Pitjor	$O(f + t)$	Totes les iteracions sempre s'executen
Anàlisi pràctic	$O(55 - 60)$	Típicament $f \approx 5 - 10$, $t \approx 20 - 50$

Complejitat espacial

$$S(f, t) = O(f + t)$$

Causada per l'emmagatzematge de nodes i aristes en memòria.

Temps pràctics

Fitxer	Línies	Temps (ms)
--------	--------	------------



fases.csv	5	0.8
transicions.csv	20	1.2
Total startup	25	3-5

3. Detecció de fases

Ubicació: grafo.py - Línia 32

Propòsit: Trobar la transició de major risc que coincideixi amb la comanda actual, sense retrocedir en el grafo de fases.

```
=====py=====
def trobar_millor_fase(input_usuari, fase_actual, totes_fases):
    input_net = input_usuari.lower()
    millor = None

    # Iterar sobre TOTS els nodes del grafo
    for node in totes_fases.values(): # O(f)
        # Iterar sobre les rutes (transicions) de cada node
        for clau, seguent in node.rutes.items(): # O(t_max)
            # Comprobacio substring: O(c) sent c = len(clau)
            if clau in input_net and seguent.risc > fase_actual.risc:
                if millor is None or seguent.risc > millor.risc:
                    millor = seguent

    return millor
=====
```

Complexitat teòrica

$$T(f, t, c) = O(f \times t_{avg} \times c)$$

Simplificat:

$$T(|E|, c) = O(|E| \times c)$$

On:

- f = nombre total de fases
- t_{avg} = transicions mitjana per fase
- c = longitud mitjana de les claus (comandes)
- $|E|$ = nombre total d'aristes al grafo

Justificació

Bucle extern: Itera sobre tots els nodes $f \rightarrow O(f)$



Bucle intern: Itera sobre les transicions de cada node $t_{avg} \rightarrow O(t_{avg})$ per node

Comprobació "clau in input_net": Cerca de substring $O(c)$ en el pitjor cas

Total: $O(f) \times O(t_{avg}) \times O(c) = O(f \times t_{avg} \times c)$

Casos de Cvmplexitat

Tipus	Complejitat	Descripció
Millor cas	$O(1)$	Primera clau coincideix i és de major risc
Cas mitjà	$O(f \times t_{avg})$	Cerca substring és ràpida
Pitjor cas	$O(f \times t_{avg} \times c)$	Totes les claus es cerquen completament

Analisis Pràctic

Paràmetre	Valors	Operacions	Temps (ms)
f (fases)	10	-	-
t_{avg} (transicions/fase)	3	30 iteracions	-
c (longitud comanda)	5	-	0.5-1.0
Total	-	150	0.5-1.0

El coll de botella de l'operació dominant és la cerca de substring clau in input_net, que utilitza l'algoritme de búsqueda de Python.

Optimitzacions Possibles

Trie (Prefix Tree): Reduir búsqueda substring a $O(\log t)$ vs $O(c)$

Índex invers (Inverted Index): Pre-indexar claus per a búsqueda ràpida

Caché de resultats: Guardar resultats freqüents de transicions

4. Lectura de CSV

Funcions afectades

"parse_log()" (monitor.py)

"_leer_bbdd()" (enriquecedor.py)

"_leer_transiciones()" (grafo.py)

"_leer_fases()" (grafo.py)

Complejitat teòrica

$T(L) = O(L)$

On L = nombre de línies en el fitxer CSV.

Justificació

Cada línia del CSV es llegeix exactament una vegada. El mòdul `csv.DictReader()` és un iterador seqüencial que processa el fitxer línia a línia sense búsquedes aleatòries ni salts de posició.

Costos Dominants

- I/O de disc: $\approx 1 - 5$ ms per fitxer (dominant)
- Parsing CSV: $\approx 0.1 - 0.5$ ms per 100 línies (negligible vs I/O)
- Inserció a hash table: $O(1)$ amortitzat per element

Optimitzacions Possibles

1. Caché en memòria: Mantenir CSV carregats en memòria entre execucions (ja parcialment implementat)
2. Compensió: Usar fitxers gzip per a reducció de mida
3. Lectura lazy: Carregar fitxers només cuando siguin necessaris



Benchmarks

Entorn de Test

- Hardware: ROG Strix G15 (Intel i7, 16GB RAM)
- SO: Arch Linux + Hyprland
- Kernel: Linux 6.x

Inicialització

Operació	Temps (ms)
Carregar fases.csv (5 files)	0.8
Carregar transicions.csv (20 files)	1.2
Carregar patrons.csv (3 files)	0.5
Total startup	3-5

Execució de Comanda Simple

Operació	Temps (ms)
Búsqueda en diccionari binarios (O(1)O(1) O(1))	0.05
Instanciació classe Comando	0.3-0.5
Execució (executar())	0.2-0.4
Total comanda simple (ls, pwd)	1-2

$$T_{fase} \approx 0.5 - 1.0 \text{ ms}$$

Causada per:

10 fases × 3 transicions mitjana = 30 iteracions

Búsqueda substring en input (~50 caracters)

Escanejo de Patrons Complet

Operació	Temps (ms)
3 patrons × historial 50 comandes	5-8
Sense optimitzacions	10-15
Amb memoization	2-3

Monitor (render)

Operació	Temps (ms)
Lectura log	0.5
detect_phase()	1.0
read_file() (credencials)	0.2
count_dir() (5 directoris)	1-2
Total render (cada 2s)	5-10

Conclusió sobre temps:

El sistema és reactiu per a sessions fins a 1000⁺ comandes. El bottleneck real és I/O de disc (0.5-1 ms per operació) no la CPU. La latència de resposta és imperceptible per a l'usuari.



Escalabilitat i límits

Factors de Creixement

Nombre de Fases (f)

Cost de muntar grafo = $O(f)$

Escalabilitat:

$f = 10$: 3-5 ms

$f = 50$: 10-15 ms

$f = 100$: 25-30 ms

$f = 500$: 150-200 ms (degradació perceptible)

Nombre de transicions (t)

Cost de detecció = $O(t)$

Escalabilitat:

$t = 20$: 0.5 ms

$t = 100$: 2-3 ms

$t = 500$: 10-15 ms (degradació clara)

Longitud de l'historial (n)

Cost escanejo patrons = $O(n \times m)$

Escalabilitat:

$n = 100$: 1-2 ms

$n = 500$: 5-10 ms

$n = 2000$: 30-50 ms (acceptable)

$n = 10000$: 150-300 ms (degradació notada)

Recomendacions d'escalabilitat

Per a ús en producció amb múltiples honeypots:

Màxim $f \approx 100$ fases

Màxim $t \approx 500$ transicions

Màxim $n \approx 5000$ comandes per sessió

Caché CSV en memòria permanent (ja implementat parcialment)



Propostes de optimització

Algoritme Aho-Corasick per a Patrons Múltiples

Problema actual:

Buscar múltiples patrons amb "caçar_patro()" té complexitat $O(P \times n \times m)$ on P = nombre de patrons.

Solució:

Implementar Aho-Corasick reduction a:

$$T_{AC} = O(n + m + z)$$

on z = nombre de matches detectats.

Guanyances:

Per a $P = 10$ patrons, $n = 500$ comandes, $m = 3$ longitud promedia:

- Actual: $O(10 \times 500 \times 3) = 15000$ operacions
- Aho-Corasick: $O(500 + 30 + z) \approx 530$ operacions
- Millora: 28x més ràpid

Trie per a búsqueda de transicions

Problema actual:

Búsqueda substring en "trobar_millor_fase()" és $O(f \times t \times c)$.

Solució:

Pre-procesar transicions en estructura Trie:

$$T_{trie} = O(f \times t + c)$$

Guanyances:

Per a $f = 10$, $t = 3$, $c = 10$:

Actual: $O(10 \times 3 \times 10) = 300$ operacions

Trie: $O(300 + 10) = 310$ (similar en aquest cas)

Millora significativa per a t grans (>20 transicions)

Caché LRU per a resultats de detecció

Implementació Usar functools.lru_cache en funcions de detecció:

```
=====
[py]=====
from functools import lru_cache

@lru_cache(maxsize=256)
def trobar_millor_fase(input_usuari, fase_id, totes_fases_tuple):
    # ... implementació ...
    pass
=====
```

Benefici:

- Historials repetitius (bots) es detecten instantàniament
- Reducció de CPU en patrons iteratius
- Caché overhead negligible



Conclusions

Conclusions Principals

1. Arquitectura Escalable: El disseny basat en un graf de decisions permet afegir dinàmicament noves fases i connexions. La complexitat temporal de $O(f \times t)$ és òptima per a sistemes de petita i mitjana escala (fins a ≈ 100 fases).
2. Algorismes Eficients: Malgrat que la funció caçar_patro() opera sota un pitjor cas de tipus polinomial $O(n \times m)$, a la pràctica és altament eficient per a cadenes curtes ($m < 5$) sota historials mitigats ($n < 500$). L'escalabilitat futura es pot blindar amb l'autòmat d'Aho-Corasick.
3. L'E/S com a Coll d'Ampolla: La taxa de lectura dels fitxers CSV representa el 80-90% del temps total del startup, desplaçant la CPU com a factor crític de rendiment.
4. Reactivitat Assegurada: En sessions d'auditoria estàndard (< 1000 comandes), el sistema ofereix una latència de resposta imperceptible (< 2 ms per comanda).
5. Camins d'Optimització Clars: La indexació per Trie, l'ús d'autòmats de cerca i la inclusió de memòries cau LRU es postulen com els candidats idonis per a una futura fase de producció.

Veredict Final

Tatami Honeypot demostra que el desplegament d'un honeypot interactiu basat en màquines d'estats és un enfocament viable i algorísmicament robust. La complexitat de les seves funcions crítiques roman dins dels paràmetres tolerables per a l'escala actual. Les optimitzacions proposades es poden introduir de manera incremental sense requerir una refactorització estructural del codi font.

Recomanació: El programari està llest per al seu desplegament en entorns acadèmics i educatius. Per a escenaris de producció complexos (més de 100 honeypots distribuïts), es recomana implementar:

1. L'algorisme d'Aho-Corasick per a la detecció de patrons maliciosos.
2. Memòria cau LRU a les funcions de transició d'estats.
3. Centralització de dades en memòria mitjançant motors indexats (Redis o PostgreSQL).